

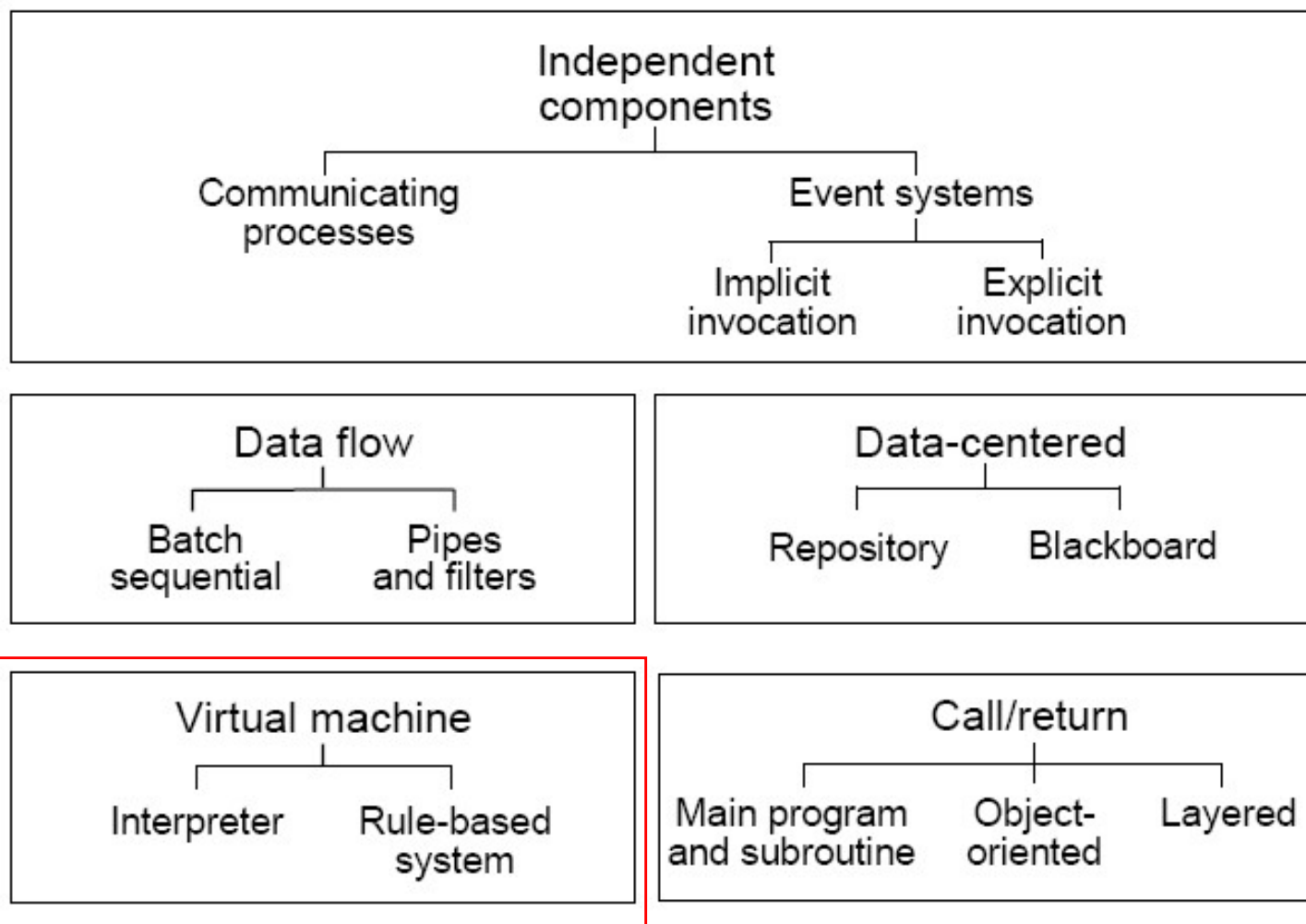


软件体系结构

第8章 解释器与基于规则

赵庆玲

南京理工大学计算机科学与工程学院



Java最吸引人的特性是什么



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

你认为**JAVA**最吸引人的特性是什么？



Object Oriented

Automatic Memory Management

Security

Built-in Networking

Robust

Compiler/Interpreter Combo

Write Once, Run Anywhere

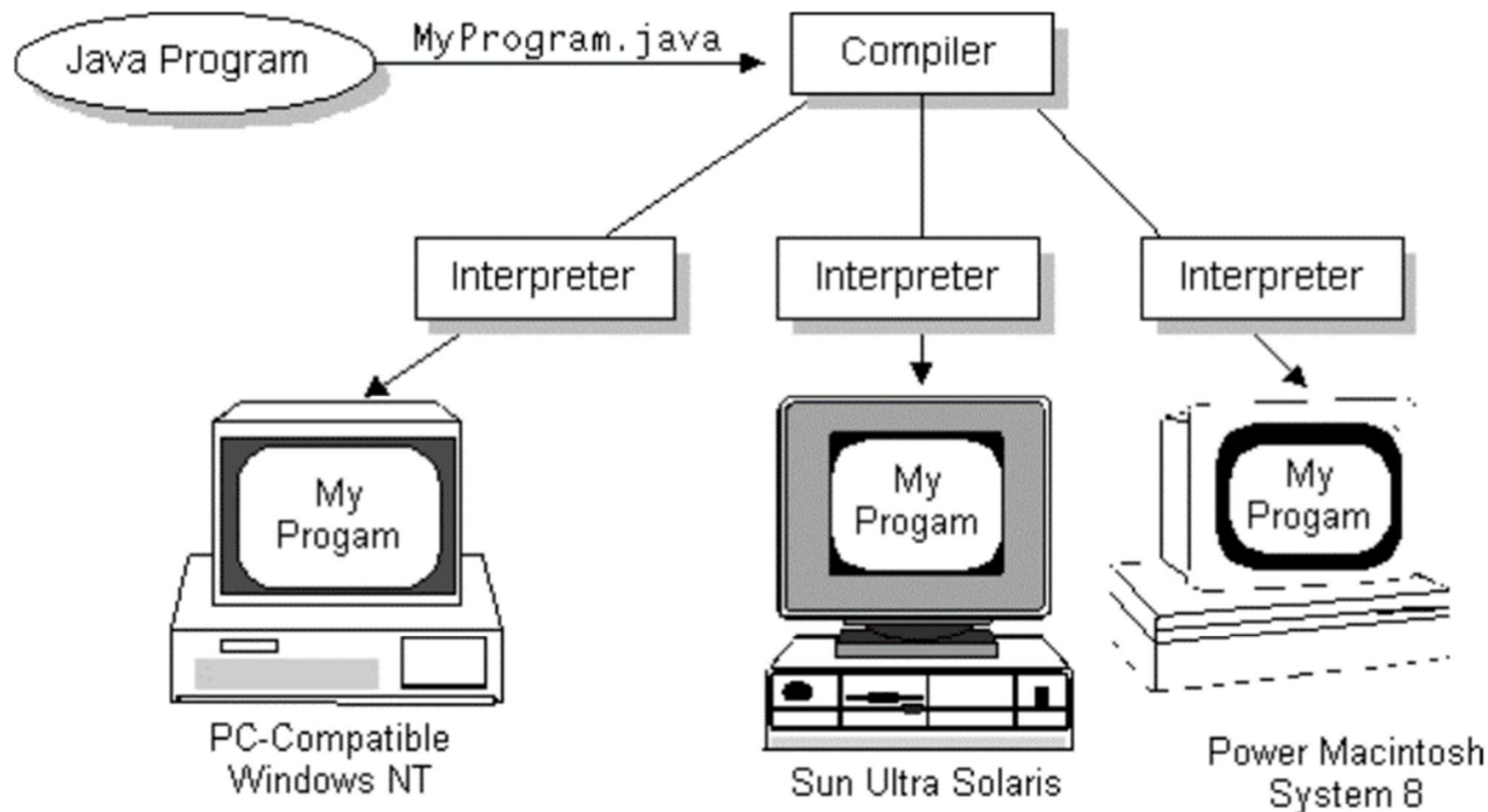
Dynamic Binding

Platform Independence

Good Performance

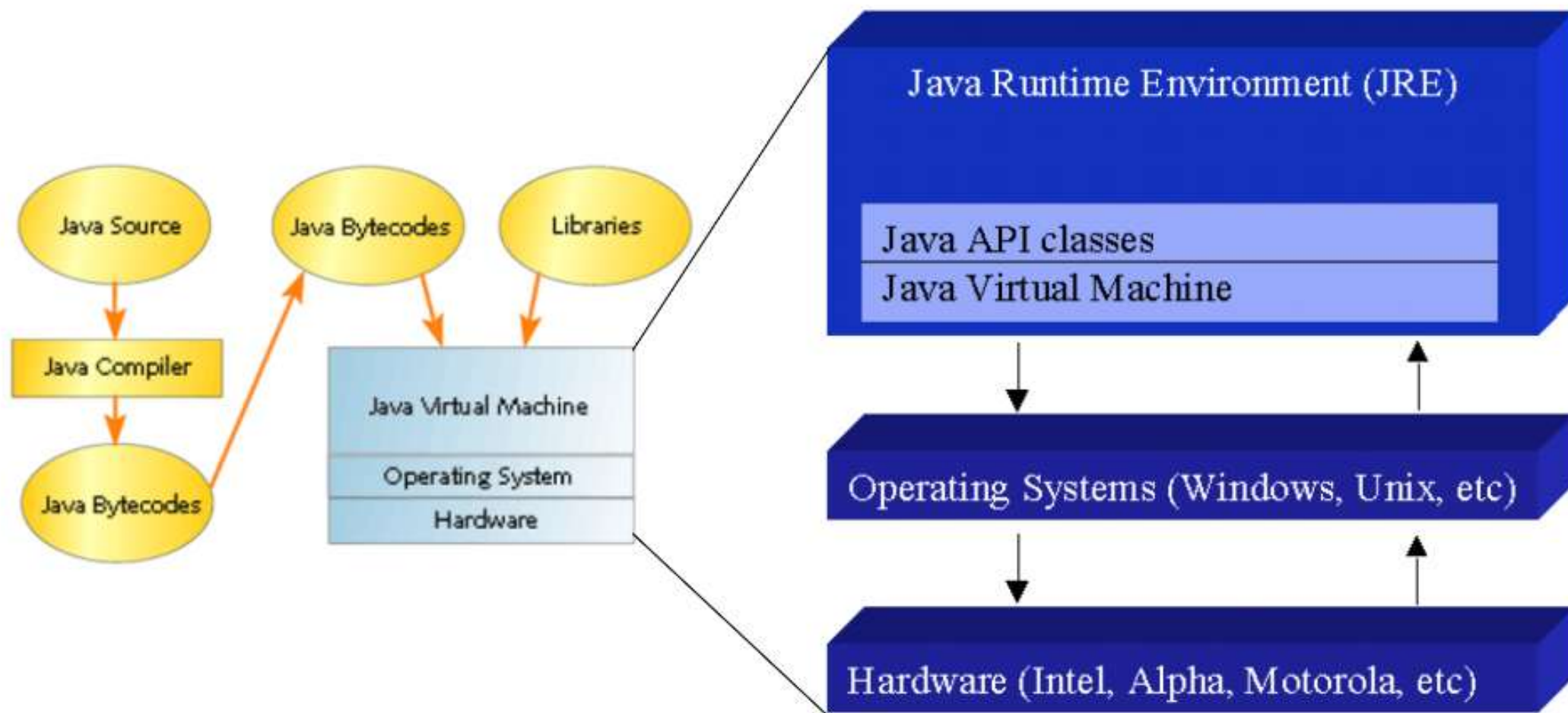
Several dangerous features of C&C++ eliminated

一次书写，多次运行





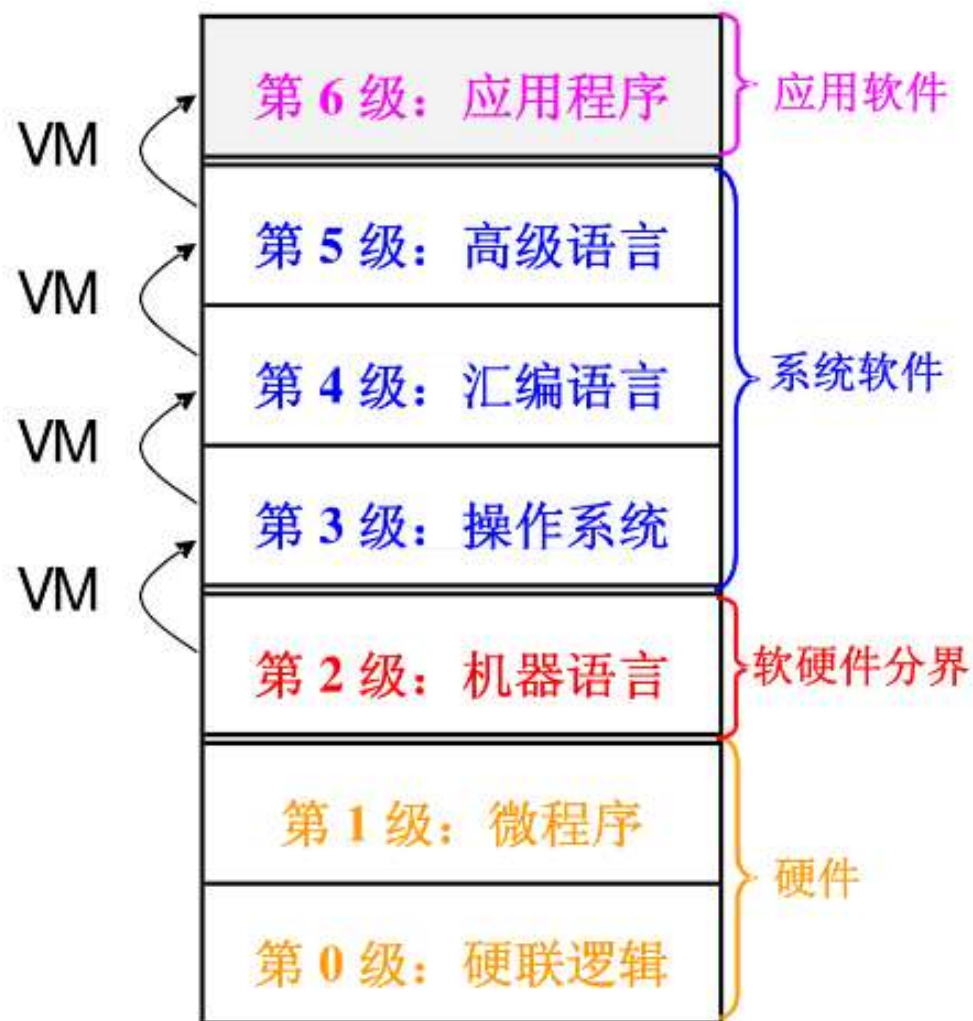
■ Java虚拟机



虚拟机



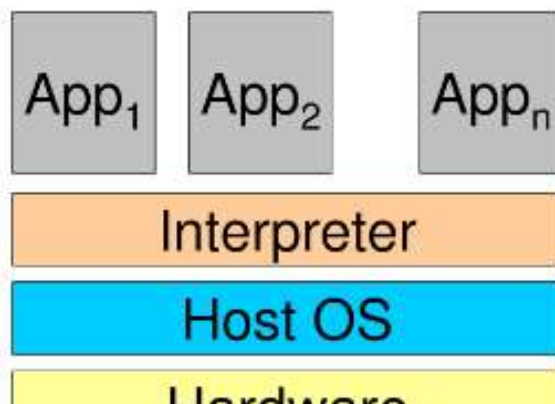
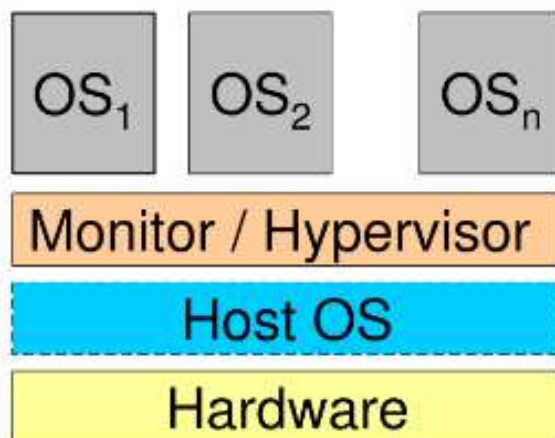
- A virtual machine is software that creates a virtualized environment between the computer platform and the end user in which the end user can operate software.
- 虚拟机是一种软件;
- 它创建了一种虚拟的环境;
- 将用户与底层平台隔离开来。



虚拟机分类



- **系统级的虚拟机(System virtual machine):** 对**OS**的虚拟, 把一台物理上独立的机器虚拟为多个不同的虚拟机, 每个虚拟机可支持运行各自的操作系统, 本质: 在**OS**与硬件之间建立隔离。
 - Hardware virtual machine (硬件虚拟机)
 - Virtual environment (虚拟环境)
 - Machine aggregation (计算机聚合)
- **进程级的虚拟机(Process virtual machine):** 对单一程序的虚拟, 其本质是在应用程序与**OS**之间建立隔离。
 - Application virtual machine (应用程序虚拟机)



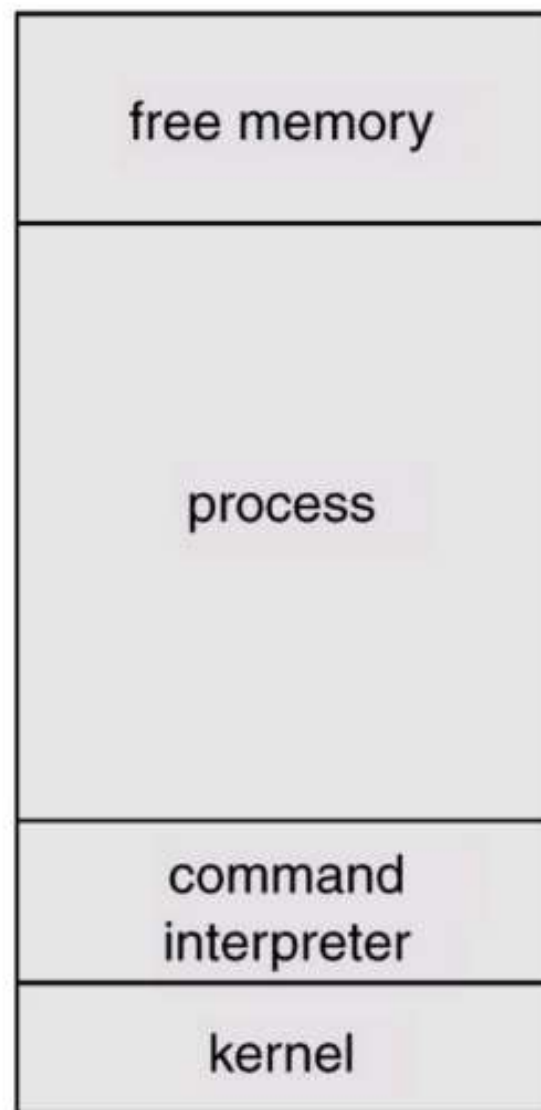
MS-DOS的命令解释器



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY



(a)



(b)

用户的命令行请求
(e.g., `dir *.jsp /a /p`)



操作系统内核的
执行指令?



- 基本思想

- 解释器(Interpreter) 是一个用来执行其他程序的程序，它针对不同的硬件平台实现了一个虚拟机，将高抽象层次的程序翻译为低抽象层次所能理解的指令，以弥补程序语义所期望的与硬件提供的计算引擎之间的差距。

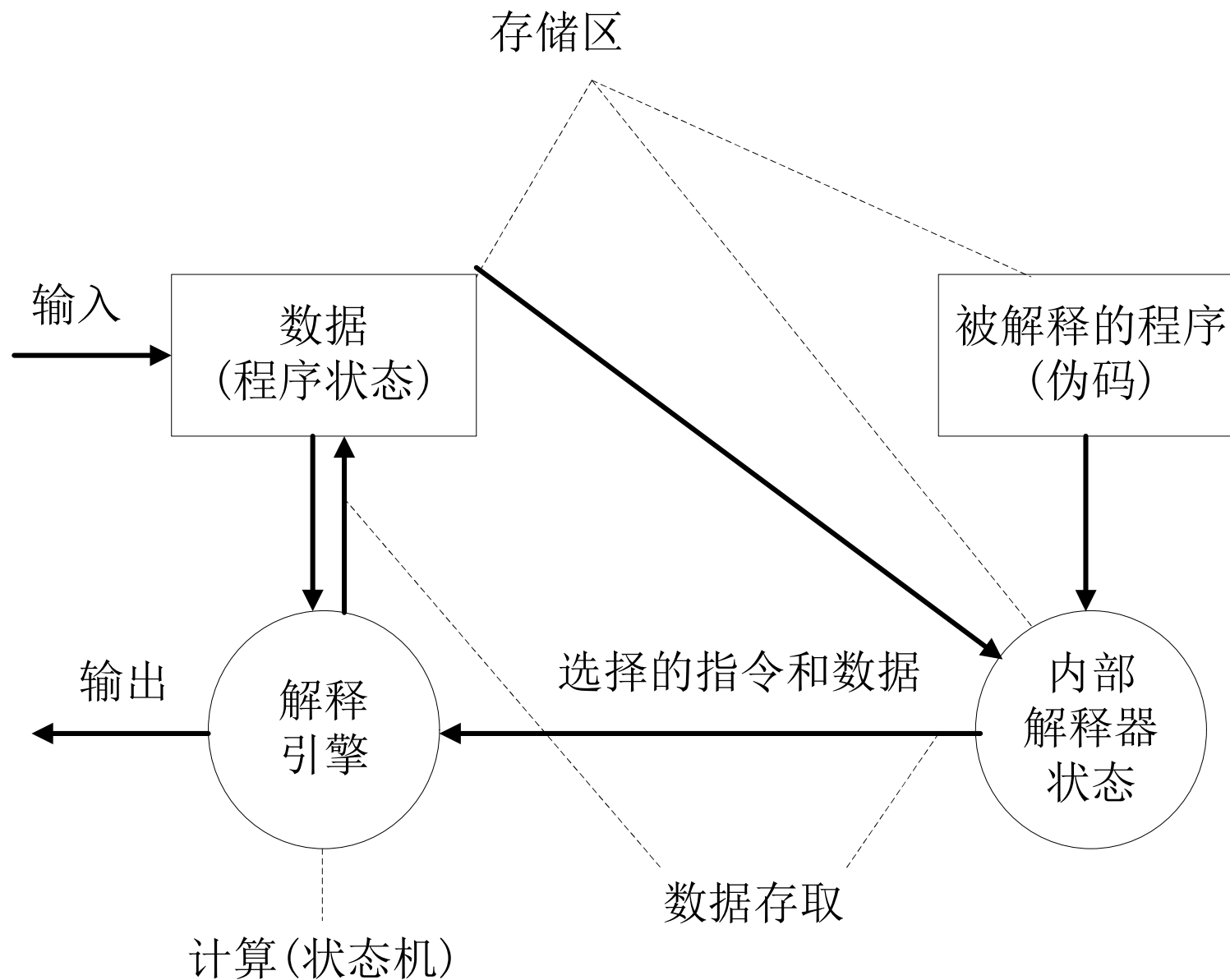


解释执行过程

组成部分



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY



- 基本构件：
 - An interpretation engine to do the work (解释器引擎)
 - A memory that contains (存储区):
 - The pseudo-code to be interpreted (被解释的源代码)
 - A representation of the control state of the interpretation engine (解释器引擎当前的内部控制状态的表示：在某个时刻需要执行哪些指令)
 - A representation of the current state of the program being simulated. (程序当前执行状态的表示)

连接器：

- Data access (对存储区的数据访问)



与解释器相对的编译器

- This is in contrast to a compiler which does not execute its input program (the source code) but translates it into another language, usually executable machine code (also called object code) which is output to a file for later execution. (编译器不会执行输入的源程序代码，而是将其翻译为另一种语言，通常是可执行的机器码或目标码，并输出到文件中以便随后链接为可执行文件并加以执行)



编译器与解释器



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

编译器

编译

链接

执行

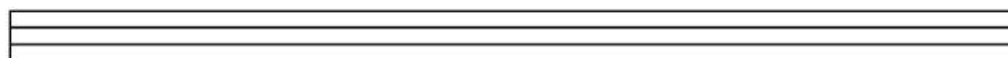


分析程序结构
源代码→目标码
确定内存分配与访问方案

分配内存
执行目标码(机器码)

解释器

解释执行



分析程序结构
确定内存分配与访问方案
分配内存
解释并执行程序

- 优点
 - （1）它有利于实现程序的可移植性和语言的跨平台能力；
 - （2）可以对未来的硬件进行模拟和仿真，能够降低测试所带来的复杂性和昂贵花费
- 缺点
 - 额外的间接层次导致了系统性能的下降，如在不引入JIT（Just In Time）技术的情况下，Java应用程序的运行速度相当慢。
 - 每次执行要分析程序结构
 - 内存分配在解释的时候进行

- 解释型语言
 - VB、Javascript、VBScript、HTML、Java字节码、Matlab
 - 脚本、配置文件
- 通信协议
- 用户输入
 - 游戏中的组合按键

- 解释器直接读取源代码并加以执行
 - –ASP
 - –Excel
 - –JavaScript
 - –MATLAB
 - –etc



- 在该类解释器下，源代码首先被“编译”为高度压缩和优化的字节码，但并不是真正的机器目标代码，因而与硬件平台无关；

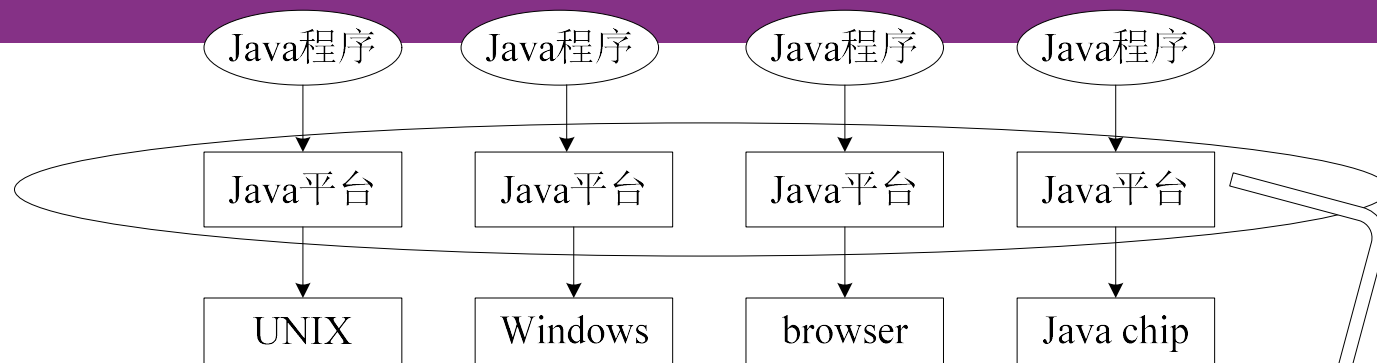
编译后得到的字节码然后被解释器加以解释；

- -Java
- -Perl
- -PHP
- -Python
- -etc

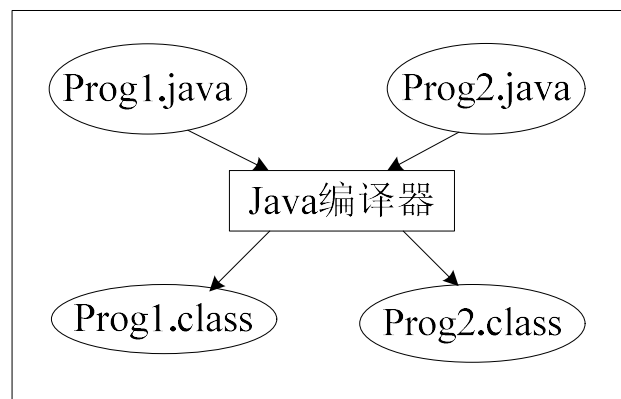
应用实例（JVM）



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

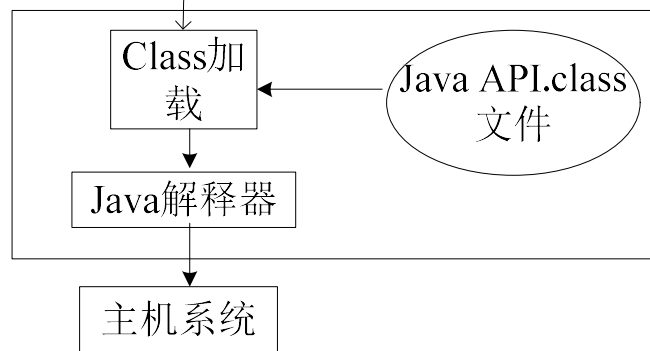


编译时环境



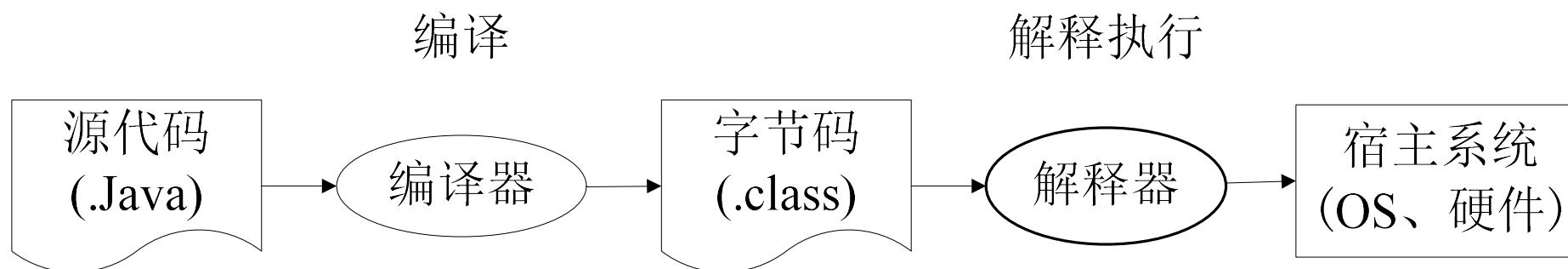
二进制字节码本地文件系统或网络间移动

运行时环境
(java平台)



基于JVM的
Java程序执行过程

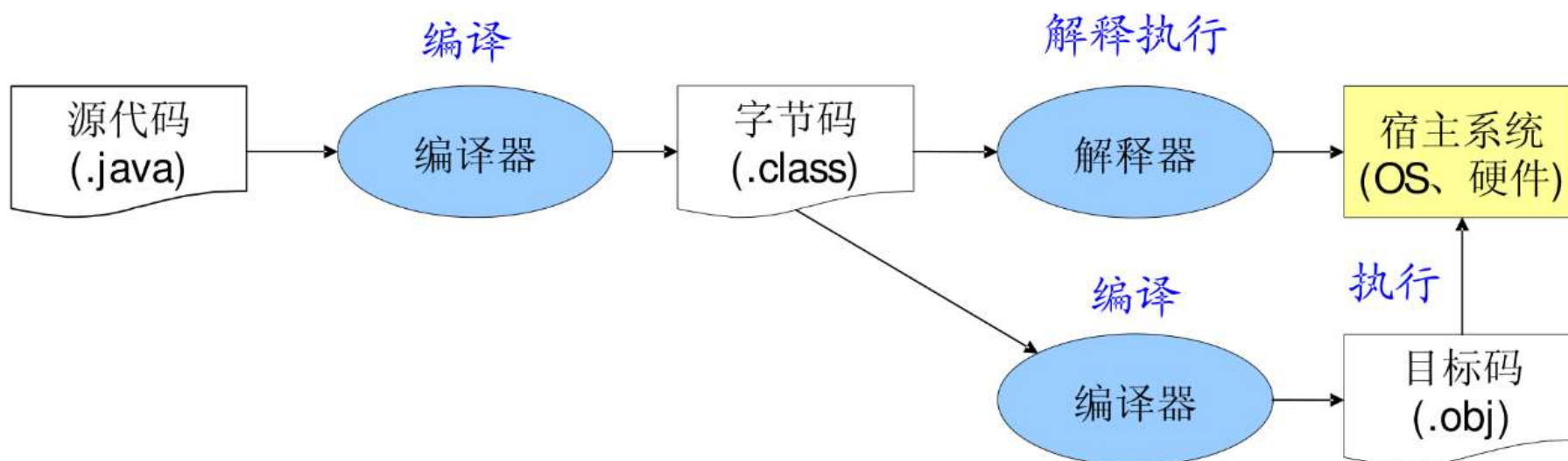
- java class文件在JVM下通过解释器运行



通过解释器执行.class文件

- Just-in-time compilation (JIT), refers to a technique where bytecode is compiled to native machine code at runtime (实时编译JIT中，字节码在运行时被编译为本机的目标代码)
 - In a JIT environment, bytecode compilation is the first step, reducing source code to a portable and optimizable intermediate representation. (第一步是编译得到字节码)
 - The bytecode is deployed onto the target system. (字节码被配置到目标系统中)
 - When the code is executed, the runtime environment's compiler translates it into native machine code. (当字节码被执行时，运行环境下的编译器将其翻译为本地机器码)
- It has gained attention in recent years, which further blurs the distinction between interpreters, byte-code interpreters and compilation. (JIT模糊了解释器、字节码解释器和编译器之间的边界与区分)

- 应用实例java class文件在JVM下通过JIT（just-in-time）编译器运行



通过实时编译方式执行.class文件



JIT编译什么？

- This can be done on a per-file or per-function basis: functions can be compiled only when they are about to be executed (hence the name “just-in-time”). (只有当某个函数要被执行时，才被编译，因此称为JIT)
- JIT compiles only those methods that contain frequently executed (“hot”) code: (而且，JIT并不是编译全部的代码，而是只编译那些被频繁执行的代码段)
 - methods that are called a large number of times (被执行多次的方法);
 - methods containing loops with large iteration counts (包含多次循环的方法).

- 现实问题：
- 现实里的业务需求经常频繁的发生变化，不断修改代码效率低、成本高。业务逻辑有时在需求阶段可能还未明确。

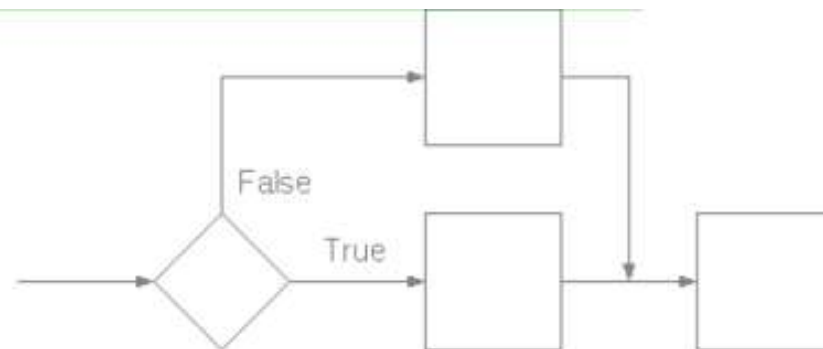
程序中的业务规则



- 程序中的业务规则，通常是以if-else分支结构的形式书写的

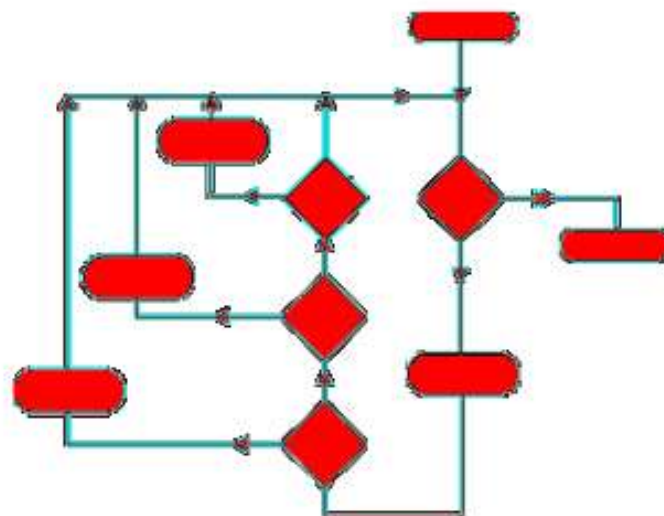
。

- 业务越复杂，判断逻辑越多，程序越难理解，越难维护...



IF-ELSE-THEN

业务规则以if-else-then的形式散落在程序代码之中



业务逻辑与代码的语义鸿沟



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 现实中的业务需求通常采用自然语言来表达，而程序员则要将其“翻译”为程序代码，二者的语法之间存在非常大的区别，从而导致“语义鸿沟”。
- 这些业务需求一旦被编写为程序源代码，就不再能被容易理解了——丢失和埋没于代码之中。
 - 想想看，用汇编语言书写的程序，哪怕再简单的业务逻辑，对大多数人来说也是不可读了。

- OO设计的原则：将可变部分和不变部分分离
 - Open-Closed Principle(开放封闭原则)
- 最好的办法是把频繁变化的业务逻辑抽取出来，形成独立的规则库。
- 这些规则可独立于软件系统而存在，可被随时更新。
- 系统运行时，读取规则库，依据当前运行状态，从规则库中选择与之匹配的规则，对规则进行解释，根据结果控制系统运行的流程。



- 基于规则系统（Rule-based System）
 - 一个使用模式匹配搜索来寻找规则并在正确的时候应用正确的逻辑知识的虚拟机。
 - 基于规则系统提供了一种基于专家系统解决问题的手段，将知识表示为“条件-行为”的规则，当满足条件时触发相应的行为，而不是将这些规则直接写在程序源代码中。



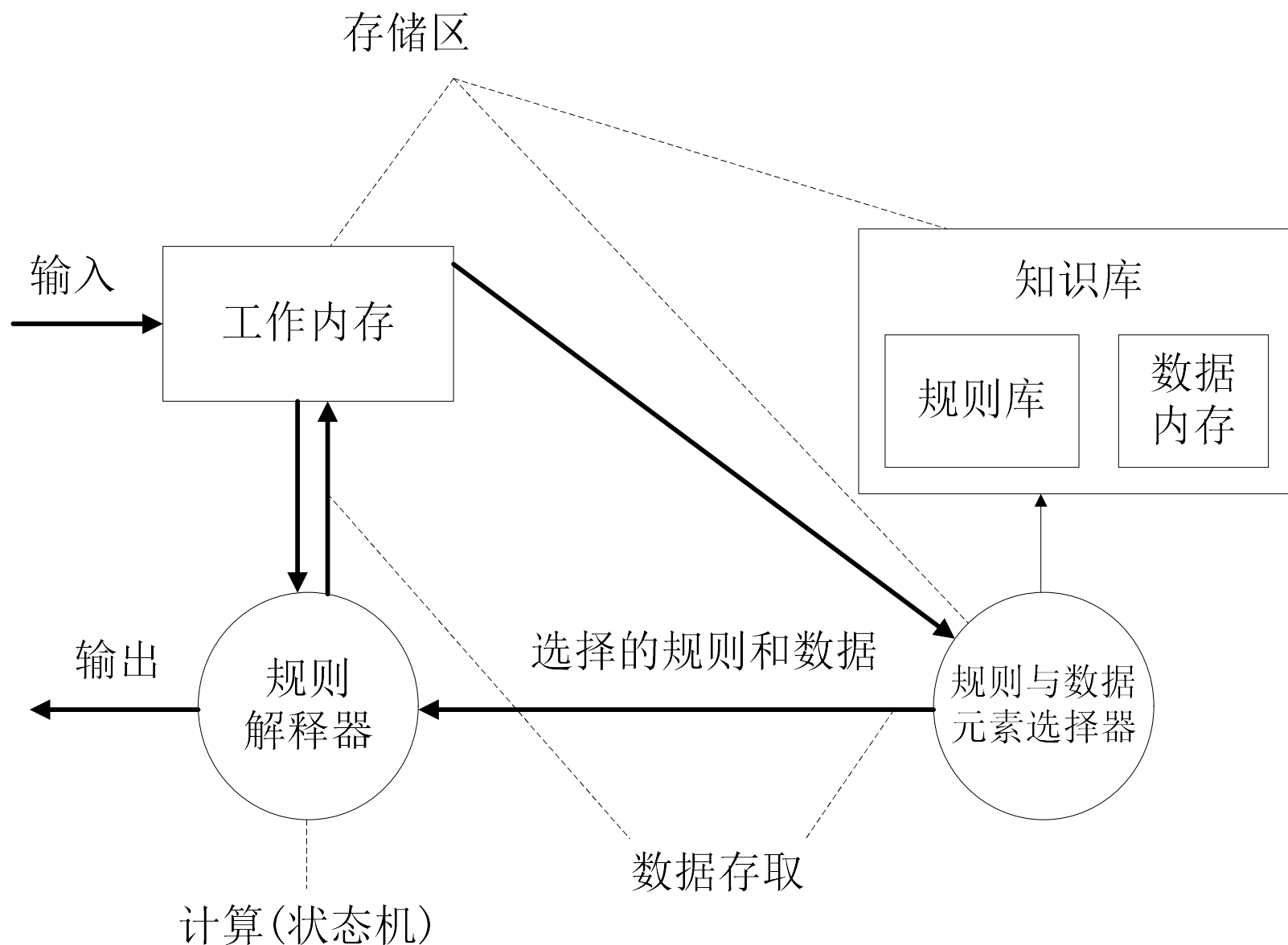
- 基于规则的系统风格的基本组件与解释器风格的组件相似

基于规则的系统	解释器风格
知识库	待解释的程序（伪码）
规则解释器	解释器引擎
规则与数据元素选择	解释器引擎内部的控制状态
工作内存	程序当前的运行状态

基于规则的系统风格



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY



基于规则的系统 and 解释器风格 的系统比较



	基于规则的系统	解释器风格的系统
相同点	基于规则的系统本质上是与解释器风格一致的，都是通过“解释器”(“规则引擎”)，在两个不同的抽象层次之间建立起一种虚拟的环境。	
不同点	在自然语言/XML的规则和高级语言的程序源代码之间建立虚拟机环境	在高级语言程序源代码和OS/硬件平台之间建立虚拟机环境

- 基于规则的系统与解释器风格的系统类似。

基于规则的系统优点



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- **Lowers the cost incurred in the modification of business logic** (降低了修改业务逻辑的成本)
- **Shortens development time** (缩短了开发时间)
- **Rules are externalized and easily shared among multiple applications** (将规则外部化, 可在多个应用之间共享)
- **Changes can be made faster and with less risk** (对规则的改变将非常迅速并且具有较低的风险)



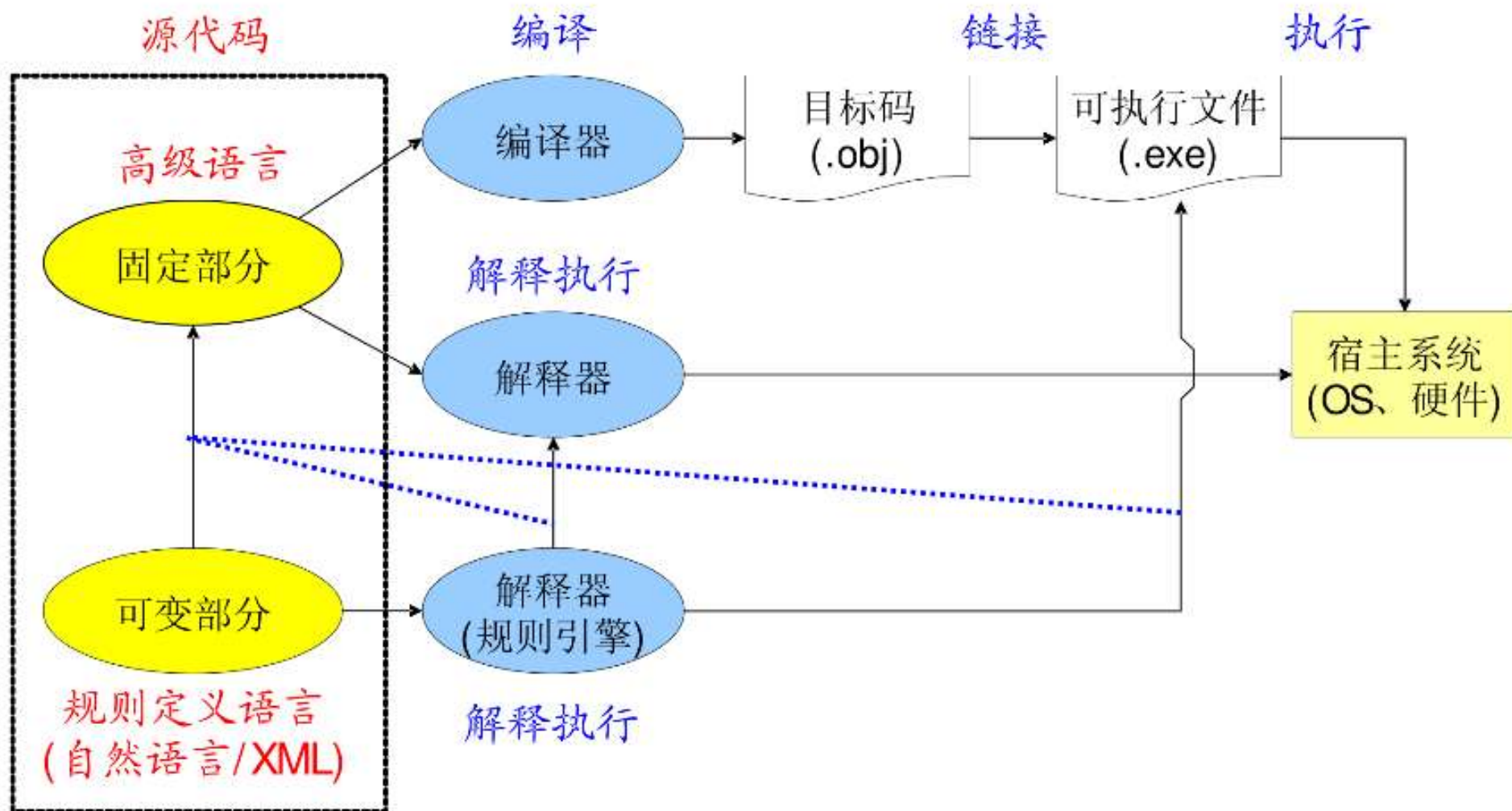
■ 核心思想:

- 将业务逻辑中可能频繁发生变化的代码从源代码中分离出来;

■ 基本过程:

- 使用规则定义语言(IF...THEN...的形式, 通常基于XML或自然语言, 但决不是程序设计语言), 将这些变化部分定义为“规则”;
- 在程序运行的时候, 规则引擎根据程序执行的当前状态, 判断需要调用并执行哪些规则, 进而通过“解释器”的方式来解释执行这些规则。
- 其他直接写在源代码里的程序, 仍然通过传统的“编译”或“解释”的办法加以执行。

执行过程



- **Any rule consists of two parts (任何规则都包含两部分) :**
 - IF part, called the antecedent (premise or condition) (IF: 规则的前提或条件)
 - THEN part, called the consequent (conclusion or action) (THEN: 规则的结论或触发的行为)
 - IF <antecedent> THEN <consequent>
- **A rule can have multiple antecedents joined by AND (conjunction), OR (disjunction) or a combination of both. (一个规则可以有多个条件, 使用AND或OR连接)**
 - IF <antecedent 1> AND (<antecedent 2> OR <antecedent 3>) THEN <consequent>

一些例子



IF Customer.age < 18
AND Withdraw.amount > 1000
THEN *signature of the parent is required*

如果客户年龄小于18岁，
而且取款金额大于1000元，
那么，需要家长的签名；

IF CustomerOrder.TotalMoney > 100
THEN CustomerOrder.Discount = 10%

如果客户订单的总金额大于100元，
那么，该订单的折扣为10%；

IF Order.Sum > ¥10,000
AND Order.Sum ≤ ¥50,000
THEN Order.AuditedBy("DepartmentManager")

如果订单总金额大于1万人民币，
而且订单总金额小于5万人民币，
那么，该订单需要由部门经理审核。

一些例子



IF ShoppingCart.purchaseAmount > \$200

THEN Customer.category = “Gold”

如果，购物车里货物的总金额大于200美元，
那么，该顾客为金卡客户；

IF Customer.category == “Gold”

THEN ShoppingCart.discount += 5%

如果，顾客为金卡客户，
那么，该客户每次购物的折扣在原有基础上提高5%；

IF ShoppingCart.discount > 15%

THEN ShoppingCart.discount = 15%

如果，某次购物的折扣超过15%，
那么，调整折扣到15%。

应用实例：基于规则的专家系统



南京理工大学
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 专家系统是一种智能的计算机程序，它能够运用知识进行推理，解决只有专家才能解决的复杂问题。
 - 是一种模拟专家决策能力的软件系统。
 - 以逻辑推理为手段，以知识为中心解决问题。
- 基于规则的专家系统：知识被表达为规则的形式。

